

Python para Backend

Componentes en

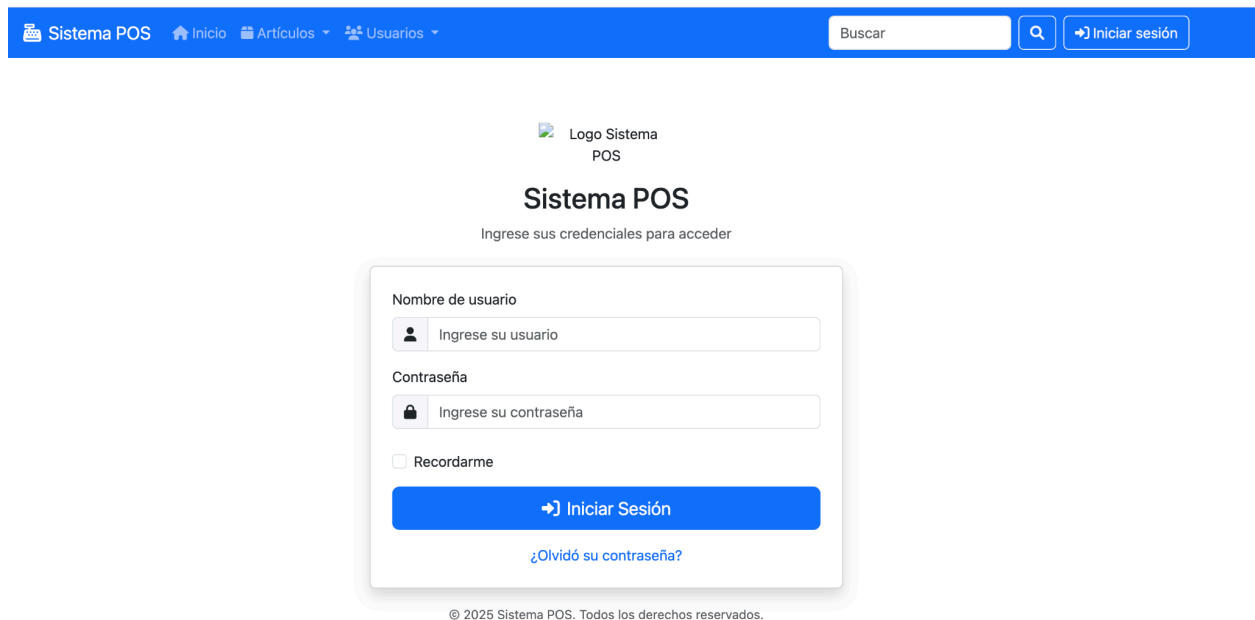
Django

Javier Villegas

05 de mayo 2025

Introducción

Para esta sesión vamos a crear un mini aplicativo web para un punto de venta, para ello utilizaremos el modelo presentado en la sesión 3, nuestro website debe quedar así:



Logo Sistema POS

Sistema POS

Ingrese sus credenciales para acceder

Nombre de usuario

Contraseña

☐ Recordarme

[Iniciar Sesión](#)

[¿Olvidó su contraseña?](#)

© 2025 Sistema POS. Todos los derechos reservados.

En esta sesión introductoria vamos a ver los siguientes temas:

1. Templates en Django
2. Archivos Estáticos
3. Vistas
4. Formularios

Implementación del Carrito de Compras

Paso 1: Crear modelos para el carrito de compras

Para nuestro carrito de compras, es necesario implementar un par de modelos más para ello vamos a crear en el archivo `pos_project/choices.py` la clase EstadoOrden:

Python

```
class EstadoOrden(models.IntegerChoices):  
    PENDIENTE = 1, "Pendiente"  
    PROCESANDO = 2, "Procesando"  
    COMPLETADA = 3, "Completada"  
    CANCELADA = 4, "Cancelada"
```

Luego en el archivo `core/models.py` agregamos la referencia a la clase EstadoOrden

Python

```
from pos_project.choices import EstadoEntidades, EstadoOrden
```

Después, vamos a crear los modelos OrdenCompraCliente y ItemOrdenCompraCliente en el archivo `core/models.py`

Python

```
class OrdenCompraCliente(models.Model):  
    pedido_id = models.UUIDField(primary_key=True, default=uuid.uuid4,  
                                editable=False)  
    nro_pedido = models.BigAutoField(unique=True, null=False,  
                                    auto_created=True)
```

```

    fecha_pedido = models.DateField(auto_now_add=True, null=False)

    cliente = models.ForeignKey('Cliente', on_delete=models.RESTRICT,
                                null=False)

    vendedor = models.ForeignKey('Vendedor', on_delete=models.RESTRICT,
                                null=False)

    importe = models.DecimalField(max_digits=10, decimal_places=2, default=0)

    estado = models.IntegerField(choices=EstadoOrden,
                                default=EstadoOrden.PENDIENTE)

    notas = models.TextField(blank=True, null=True)

    creado_por = models.ForeignKey(Usuario, on_delete=models.RESTRICT,
                                null=False)

    fecha_creacion = models.DateTimeField(auto_now_add=True, null=False)

    def actualizar_total(self):
        """Actualiza el total de la orden basado en los items"""
        total = sum(item.total_item for item in self.items_orden_compra.all())
        self.importe = total
        self.save()

    def __str__(self):
        return f"Orden #{self.nro_pedido} - {self.cliente}"

    class Meta:
        db_table = "ordenes_compra_cliente"
        ordering = ["-fecha_creacion"]

class ItemOrdenCompraCliente(models.Model):

```

```

        item_id = models.UUIDField(primary_key=True, default=uuid.uuid4,
editable=False)

        pedido = models.ForeignKey(OrdenCompraCliente, on_delete=models.CASCADE,
null=False, related_name='items_orden_compra')

        nro_item = models.PositiveIntegerField(default=1, null=False)

        articulo = models.ForeignKey('Articulo', on_delete=models.RESTRICT,
null=False, related_name='articulo_item_orden_compra')

        cantidad = models.PositiveIntegerField(null=False, default=1)

        precio_unitario = models.DecimalField(max_digits=12, decimal_places=2,
null=False, default=0)

        total_item = models.DecimalField(max_digits=12, decimal_places=2,
null=False, default=0)

        estado = models.IntegerField(choices=EstadoEntidades,
default=EstadoEntidades.ACTIVO)

        creado_por = models.ForeignKey(Usuario, on_delete=models.RESTRICT,
null=False)

        fecha_creacion = models.DateTimeField(auto_now_add=True, null=False)

def save(self, *args, **kwargs):

    # Calcular el total del item

    self.total_item = self.cantidad * self.precio_unitario

    # Si no se ha establecido el precio unitario, tomarlo del artículo

    if self.precio_unitario == 0:

        try:

            lista_precio = self.articulo.listaprecio

            self.precio_unitario = lista_precio.precio_1

            self.total_item = self.cantidad * self.precio_unitario

```

```
except:
    pass

super().save(*args, **kwargs)

# Actualizar el total de la orden
self.pedido.actualizar_total()

def __str__(self):
    return f"{self.cantidad} x {self.articulo.descripcion}"

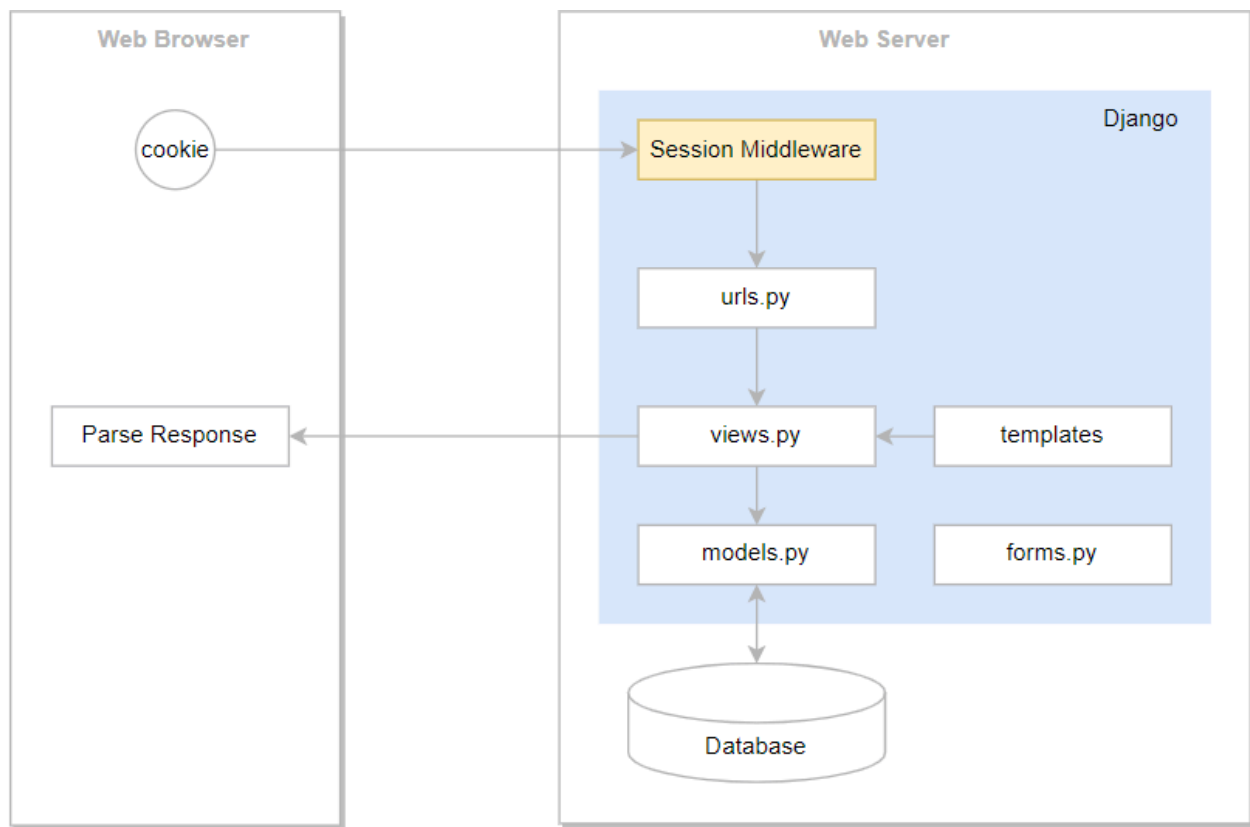
class Meta:
    db_table = "items_ordenes_compra_cliente"
```

Después de haber creado los respectivos modelos debemos de aplicar y ejecutar nuestras migraciones.

Paso 1: Implementar el carrito de compras usando sesiones

Sesiones en Django

Django proporciona soporte completo para sesiones anónimas. El marco de sesiones te permite almacenar y recuperar datos arbitrarios por visitante. Mantiene los datos en el lado del servidor y abstrae el envío y recepción de cookies.



Cuando un navegador web realiza la primera petición HTTP al servidor web, el middleware de sesión inicia una nueva sesión. El middleware de sesión genera un número grande y aleatorio que se denomina identificador de sesión y lo envía al navegador web en forma de cookie.

Para las siguientes peticiones, el middleware de sesión compara el valor sessionid de la cookie enviada por el navegador web con el identificador de sesión almacenado en el servidor web y asocia los datos de sesión con el objeto de petición HTTP.

La información sobre las sesiones actuales y caducadas debe almacenarse en algún lugar. En los primeros días de la web, esto se hacía guardando la información de sesión en archivos en el servidor, pero a medida que las arquitecturas de los servidores web se han vuelto más elaboradas y sus demandas de rendimiento han aumentado, otras estrategias más eficientes como una base de datos o el almacenamiento en memoria se han convertido en la norma. **Por defecto, en Django, la información de sesión se almacena en la base de datos de un proyecto.**

Este es un valor por defecto razonable para la mayoría de los proyectos pequeños. Sin embargo, la implementación del middleware de sesiones de Django nos da la flexibilidad de almacenar la información de sesión de nuestro proyecto en una variedad de formas para adaptarse a la arquitectura de nuestro sistema y los requisitos de rendimiento. Cada una de estas diferentes

implementaciones se denomina motor de sesión. Si queremos cambiar la configuración de la sesión, necesitamos especificar el ajuste `SESSION_ENGINE` en el archivo `settings.py` del proyecto:

- **Sesiones en caché:** En algunos entornos, el almacenamiento en caché de la información de sesión en memoria o en una base de datos es un enfoque adecuado para obtener un alto rendimiento. Django proporciona los motores de sesión `django.contrib.sessions.backends.cache` y `django.contrib.sessions.backends.cached_db` para este propósito.
- **Sesiones basadas en archivos:** Como se ha dicho antes, esta es una forma algo anticuada de mantener la información de sesión, pero puede ser adecuada para algunos sitios en los que el rendimiento no es un problema y hay razones para no almacenar información dinámica en una base de datos.
- **Sesiones basadas en cookies:** En lugar de mantener la información de sesión en el lado del servidor, puede mantenerlas completamente en el cliente del navegador web serializando el contenido de la sesión como JSON y almacenándolo en una cookie basada en el navegador.

Ahora para nuestro ejemplo vamos a empezar a crear nuestras sesiones, para ello vamos a crear una clase para gestionar un carrito de compras usando sesiones, para ellos vamos a crear el archivo `core/cart.py` y escribimos lo siguiente:

Python

```
# Crear archivo core/cart.py

from decimal import Decimal
from django.conf import settings
from .models import Artículo

class Cart:

    """

    Clase para gestionar el carrito de compras mediante sesiones

    """
```

```

def __init__(self, request):
    """
    Inicializa el carrito
    """
    self.session = request.session
    cart = self.session.get('cart')
    if not cart:
        # Crear un carrito vacío si no existe
        cart = self.session['cart'] = {}
    self.cart = cart

def add(self, articulo, cantidad=1, update_cantidad=False):
    """
    Añadir un producto al carrito o actualizar su cantidad
    """
    articulo_id = str(articulo.articulo_id)
    if articulo_id not in self.cart:
        # Obtener el precio del artículo
        try:
            lista_precio = articulo.listaprecio
            precio = float(lista_precio.precio_1)
        except:
            precio = 0

        self.cart[articulo_id] = {

```

```

        'cantidad': 0,

        'precio': precio,

        'descripcion': articulo.descripcion
    }

    if update_cantidad:

        self.cart[articulo_id]['cantidad'] = cantidad
    else:

        self.cart[articulo_id]['cantidad'] += cantidad

    self.save()

def save(self):
    """
    Guardar los cambios en la sesión
    """
    self.session.modified = True

def remove(self, articulo):
    """
    Eliminar un producto del carrito
    """
    articulo_id = str(articulo.articulo_id)
    if articulo_id in self.cart:
        del self.cart[articulo_id]

```

```

        self.save()

def __iter__(self):
    """
    Iterar sobre los elementos en el carrito y obtener los artículos de la
    base de datos
    """
    articulo_ids = self.cart.keys()
    articulos = Articulo.objects.filter(articulo_id__in=articulo_ids)

    cart = self.cart.copy()
    for articulo in articulos:
        cart[str(articulo.articulo_id)]['articulo'] = articulo

    for item in cart.values():
        item['precio'] = Decimal(item['precio'])
        item['total_precio'] = item['precio'] * item['cantidad']
        yield item

def __len__(self):
    """
    Contar todos los items en el carrito
    """
    return sum(item['cantidad'] for item in self.cart.values())

def get_total_price(self):

```

```

    """
    Calcular el costo total de los items
    """

    return sum(Decimal(item['precio']) * item['cantidad'] for item in
self.cart.values())

def clear(self):
    """
    Eliminar el carrito de la sesión
    """

    del self.session['cart']

    self.save()

```

Ahora vamos a crear las vistas para el carrito de compras, vamos al archivo core/views.py y primero agregamos esto

```

Python

from django.views.decorators.http import require_POST

from .cart import Cart

from .models import Articulo, GrupoArticulo, LineaArticulo, ListaPrecio,
Cliente, Vendedor, EstadoOrden

```

Luego vamos hacia el final de archivo y agregamos lo siguiente:

Python

```
@require_POST

def cart_add(request, articulo_id):
    """
    Añadir artículo al carrito
    """
    cart = Cart(request)
    articulo = get_object_or_404(Articulo, articulo_id=articulo_id)
    cantidad = int(request.POST.get('cantidad', 1))
    update = request.POST.get('update')

    cart.add(articulo=articulo, cantidad=cantidad, update_cantidad=update)
    messages.success(request, f'"{articulo.descripcion}" añadido al carrito.')

    return redirect('cart_detail')

def cart_remove(request, articulo_id):
    """
    Eliminar artículo del carrito
    """
    cart = Cart(request)
    articulo = get_object_or_404(Articulo, articulo_id=articulo_id)
    cart.remove(articulo)
    messages.info(request, f'"{articulo.descripcion}" eliminado del carrito.')
```

```

        return redirect('cart_detail')

def cart_detail(request):
    """
    Ver detalle del carrito
    """
    cart = Cart(request)
    return render(request, 'core/cart/detail.html', {'cart': cart})

def cart_clear(request):
    """
    Vaciar el carrito
    """
    cart = Cart(request)
    cart.clear()
    messages.info(request, 'Carrito vaciado correctamente.')

    return redirect('cart_detail')

@login_required
def checkout(request):
    """
    Finalizar compra
    """
    cart = Cart(request)

```

```

if len(cart) == 0:
    messages.warning(request, 'Tu carrito está vacío')
    return redirect('cart_detail')

# Obtener o crear cliente para el usuario
try:
    cliente = Cliente.objects.get(correo_electronico=request.user.email)
except Cliente.DoesNotExist:
    # Obtener tipo de identificación predeterminado (puedes ajustar según
    tus datos)
    try:
        tipo_id = TipoIdentificacion.objects.first()
        canal = CanalCliente.objects.first()

        # Crear cliente con datos básicos del usuario
        cliente = Cliente.objects.create(
            cliente_id=uuid.uuid4(),
            tipo_identificacion=tipo_id,
            nro_documento=request.user.username[:11], # Ejemplo
simplificado
            nombres=request.user.full_name,
            correo_electronico=request.user.email,
            canal=canal,
            estado=EstadoEntidades.ACTIVO
        )

```

```

        except:

            messages.error(request, 'Error al procesar el cliente. Contacte al
administrador.')

            return redirect('cart_detail')

# Obtener vendedor predeterminado para la orden
try:

    vendedor = Vendedor.objects.first()

except:

    messages.error(request, 'No hay vendedores disponibles. Contacte al
administrador.')

    return redirect('cart_detail')

if request.method == 'POST':

    # Crear la orden
    try:

        orden = OrdenCompraCliente.objects.create(

            pedido_id=uuid.uuid4(),

            cliente=cliente,

            vendedor=vendedor,

            estado=EstadoOrden.PENDIENTE,

            notas=request.POST.get('notas', ''),

            creado_por=request.user

        )

        # Crear los items de la orden

```

```

for item in cart:
    articulo = item['articulo']

    ItemOrdenCompraCliente.objects.create(
        item_id=uuid.uuid4(),
        pedido=orden,
        nro_item=1, # Puedes implementar una secuencia
        articulo=articulo,
        cantidad=item['cantidad'],
        precio_unitario=item['precio'],
        creado_por=request.user
    )

# Limpiar el carrito
cart.clear()

messages.success(request, f'¡Orden creada exitosamente! Tu número
de orden es: {orden.nro_pedido}')

return redirect('order_detail', pedido_id=orden.pedido_id)

except Exception as e:
    messages.error(request, f'Error al procesar la orden: {str(e)}')
    return redirect('cart_detail')

return render(request, 'core/cart/checkout.html', {
    'cart': cart,
    'cliente': cliente
})

```

```

    })

@login_required
def order_detail(request, pedido_id):
    """
    Ver detalle de una orden
    """

    orden = get_object_or_404(OrdenCompraCliente, pedido_id=pedido_id)

    # Verificar que la orden pertenece al usuario actual
    if orden.cliente.correo_electronico != request.user.email and not request.user.is_staff:
        messages.error(request, 'No tienes permiso para ver esta orden.')
        return redirect('home')

    return render(request, 'core/cart/order_detail.html', {'orden': orden})

```

Luego vamos a crear la plantilla para el carrito de compras para ello vamos primero a crear dentro de nuestra carpeta de templates, una carpeta llamada cart, y luego crear el archivo `templates/cart/detail.html` y escribimos el siguiente código:

```

Python

{% extends 'base.html' %}

{% block title %}Carrito de Compras - Sistema POS{% endblock %}

```

```

{% block content %}

<div class="d-flex justify-content-between align-items-center mb-4">

    <h2 class="h3"><i class="fas fa-shopping-cart me-2"></i>Carrito de
    Compras</h2>

    <a href="{% url 'articulos_list' %}" class="btn btn-outline-primary">

        <i class="fas fa-arrow-left me-1"></i>Continuar Comprando

    </a>

</div>

<div class="card shadow">

    <div class="card-header bg-white">

        <h5 class="mb-0">Productos en tu Carrito</h5>

    </div>

    <div class="card-body p-0">

        {% if cart|length > 0 %}

        <div class="table-responsive">

            <table class="table table-hover mb-0">

                <thead>

                    <tr>

                        <th>Producto</th>

                        <th>Precio</th>

                        <th>Cantidad</th>

                        <th>Subtotal</th>

                        <th>Acciones</th>

                    </tr>

```

```

</thead>

<tbody>

    {% for item in cart %}

    <tr>

        <td>{{ item.descripcion }}</td>

        <td>${{ item.precio }}</td>

        <td>

            <form action="{% url 'cart_add'
item.articulo.articulo_id %}" method="post" class="d-flex align-items-center">

                {% csrf_token %}

                <input type="number" name="cantidad" value="{{
item.cantidad }}" min="1" class="form-control form-control-sm" style="width:
70px;">

                <input type="hidden" name="update"
value="True">

                <button type="submit" class="btn btn-sm
btn-outline-secondary ms-2">

                    <i class="fas fa-sync-alt"></i>

                </button>

            </form>

        </td>

        <td>${{ item.total_precio }}</td>

        <td>

            <a href="{% url 'cart_remove'
item.articulo.articulo_id %}" class="btn btn-sm btn-danger">

                <i class="fas fa-trash-alt"></i>

            </a>

        </td>

    </tr>

    {% endfor %}

</tbody>

```

```

        </tr>

        {% endfor %}

    </tbody>

    <tfoot>

        <tr>

            <th colspan="3" class="text-end">Total:</th>

            <th>${{ cart.get_total_price }}</th>

            <th></th>

        </tr>

    </tfoot>

</table>

</div>

<div class="card-footer bg-white d-flex justify-content-between align-items-center">

    <a href="{% url 'cart_clear' %}" class="btn btn-outline-danger"
onclick="return confirm('¿Estás seguro de vaciar el carrito?')">

        <i class="fas fa-trash me-1"></i>Vaciar Carrito

    </a>

    <a href="{% url 'checkout' %}" class="btn btn-success">

        <i class="fas fa-check me-1"></i>Proceder al Pago

    </a>

</div>

{% else %}

<div class="text-center py-5">

    <i class="fas fa-shopping-cart fa-4x text-muted mb-3"></i>

    <h4>Tu carrito está vacío</h4>

```

```

        <p class="text-muted">Añade productos a tu carrito para continuar
comprando</p>

        <a href="{% url 'articulos_list' %}" class="btn btn-primary mt-3">

            <i class="fas fa-shopping-bag me-1"></i>Ver Productos

        </a>

    </div>

    {% endif %}

</div>

</div>

{% endblock %}

```

A continuación, nos toca crear la plantilla para el proceso de checkout del carrito, esto para cuándo se termine de hacer la compra y queramos procesar dicha compra, para ello creamos el archivo `templates/cart/checkout.html` y escribimos lo siguiente:

Python

```

{% extends 'base.html' %}

{% block title %}Finalizar Compra - Sistema POS{% endblock %}

{% block content %}

<div class="d-flex justify-content-between align-items-center mb-4">

    <h2 class="h3"><i class="fas fa-credit-card me-2"></i>Finalizar Compra</h2>

    <a href="{% url 'cart_detail' %}" class="btn btn-outline-secondary">

        <i class="fas fa-arrow-left me-1"></i>Volver al Carrito

```

```

    </a>
</div>

<div class="row">
    <div class="col-md-8">
        <div class="card shadow mb-4">
            <div class="card-header bg-white">
                <h5 class="mb-0">Datos de Facturación</h5>
            </div>
            <div class="card-body">
                <form method="post" id="checkoutForm">
                    {% csrf_token %}

                    <div class="row mb-3">
                        <div class="col-md-12">
                            <label for="nombre" class="form-label">Nombre
Completo</label>
                            <input type="text" class="form-control" id="nombre"
value="{{ cliente.nombres }}" readonly>
                        </div>
                    </div>
                </div>

                <div class="mb-3">
                    <label for="email" class="form-label">Email</label>
                    <input type="email" class="form-control" id="email"
value="{{ cliente.correo_electronico }}" readonly>
                </div>
            </div>
        </div>
    </div>

```

```

        </div>

        <div class="mb-3">
            <label for="direccion" class="form-label">Dirección de
Entrega</label>
            <textarea class="form-control" id="direccion"
name="direccion" rows="3" required>{{ cliente.direccion|default:''
}}</textarea>
        </div>

        <div class="mb-3">
            <label for="notas" class="form-label">Notas
adicionales</label>
            <textarea class="form-control" id="notas" name="notas"
rows="3"></textarea>
        </div>

        <div class="d-grid gap-2 d-md-flex justify-content-md-end">
            <button type="submit" class="btn btn-success">
                <i class="fas fa-check me-1"></i>Completar Orden
            </button>
        </div>
    </form>
</div>
</div>
</div>

```

```

<div class="col-md-4">
  <div class="card shadow mb-4">
    <div class="card-header bg-white">
      <h5 class="mb-0">Resumen de Compra</h5>
    </div>
    <div class="card-body">
      <ul class="list-group list-group-flush">
        {% for item in cart %}
          <li class="list-group-item d-flex justify-content-between
lh-sm">
            <div>
              <h6 class="my-0">{{ item.descripcion }}</h6>
              <small class="text-muted">{{ item.cantidad }} x ${{
item.precio }}</small>
            </div>
            <span class="text-muted">${{ item.total_precio
}}</span>
          </li>
        {% endfor %}
        <li class="list-group-item d-flex justify-content-between">
          <span>Total</span>
          <strong>${{ cart.get_total_price }}</strong>
        </li>
      </ul>
    </div>
  </div>

```

```

<div class="card shadow">

  <div class="card-header bg-white">

    <h5 class="mb-0">Métodos de Pago</h5>

  </div>

  <div class="card-body">

    <div class="form-check mb-3">

      <input class="form-check-input" type="radio"
name="paymentMethod" id="efectivo" checked>

      <label class="form-check-label" for="efectivo">

        <i class="fas fa-money-bill-wave me-2"></i>Efectivo

      </label>

    </div>

    <div class="form-check mb-3">

      <input class="form-check-input" type="radio"
name="paymentMethod" id="tarjeta">

      <label class="form-check-label" for="tarjeta">

        <i class="fas fa-credit-card me-2"></i>Tarjeta de

Crédito/Débito

      </label>

    </div>

    <div class="form-check">

      <input class="form-check-input" type="radio"
name="paymentMethod" id="transferencia">

      <label class="form-check-label" for="transferencia">

        <i class="fas fa-exchange-alt me-2"></i>Transferencia

Bancaria

```

```

        </label>

    </div>

</div>

</div>

</div>

</div>

{% endblock %}

```

Finalmente, vamos a crear una plantilla para poder visualizar el detalle de la orden de compra en nuestra solución web, para ello agregamos el siguiente archivo `templates/cart/order_detail.html` y escribimos lo siguiente:

Python

```

{% extends 'base.html' %}

{% block title %}Orden #{{ orden.nro_pedido }} - Sistema POS{% endblock %}

{% block content %}

<div class="d-flex justify-content-between align-items-center mb-4">

    <h2 class="h3"><i class="fas fa-file-invoice me-2"></i>Detalle de Orden #{{
orden.nro_pedido }}</h2>

    <div>

        <a href="{% url 'home' %}" class="btn btn-outline-secondary me-2">

            <i class="fas fa-home me-1"></i>Inicio

        </a>

```

```

        <a href="{% url 'generate_pdf_order' orden.pedido_id %}" class="btn
btn-primary" target="_blank">

            <i class="fas fa-file-pdf me-1"></i>Descargar PDF

        </a>

    </div>

</div>

<div class="row">

    <div class="col-md-8">

        <div class="card shadow mb-4">

            <div class="card-header bg-white">

                <h5 class="mb-0">Información de la Orden</h5>

            </div>

            <div class="card-body">

                <div class="row mb-4">

                    <div class="col-md-6">

                        <p class="mb-1 text-muted">Número de Orden:</p>

                        <h5>{{ orden.nro_pedido }}</h5>

                    </div>

                    <div class="col-md-6">

                        <p class="mb-1 text-muted">Fecha:</p>

                        <h5>{{ orden.fecha_pedido }}</h5>

                    </div>

                </div>

            </div>

        </div>

        <div class="row mb-4">

```

```

<div class="col-md-6">

    <p class="mb-1 text-muted">Cliente:</p>

    <h5>{{ orden.cliente.nombres }}</h5>

</div>

<div class="col-md-6">

    <p class="mb-1 text-muted">Estado:</p>

    <h5>

        {% if orden.estado == 1 %}

        <span class="badge bg-warning">Pendiente</span>

        {% elif orden.estado == 2 %}

        <span class="badge bg-info">Procesando</span>

        {% elif orden.estado == 3 %}

        <span class="badge bg-success">Completada</span>

        {% else %}

        <span class="badge bg-danger">Cancelada</span>

        {% endif %}

    </h5>

</div>

</div>

{% if orden.notas %}

<div class="mb-4">

    <p class="mb-1 text-muted">Notas:</p>

    <p>{{ orden.notas }}</p>

</div>

```

```

{% endif %}

<div class="table-responsive">
  <table class="table table-hover">
    <thead>
      <tr>
        <th>#</th>
        <th>Producto</th>
        <th>Precio</th>
        <th>Cantidad</th>
        <th>Total</th>
      </tr>
    </thead>
    <tbody>
      {% for item in orden.items_orden_compra.all %}
      <tr>
        <td>{{ item.nro_item }}</td>
        <td>{{ item.articulo.descripcion }}</td>
        <td>${{ item.precio_unitario }}</td>
        <td>{{ item.cantidad }}</td>
        <td>${{ item.total_item }}</td>
      </tr>
      {% endfor %}
    </tbody>
    <tfoot>

```

```

        <tr>

            <th colspan="4" class="text-end">Total:</th>

            <th>${{ orden.importe }}</th>

        </tr>

    </tfoot>

</table>

</div>

</div>

</div>

</div>

<div class="col-md-4">

    <div class="card shadow mb-4">

        <div class="card-header bg-white">

            <h5 class="mb-0">Información del Cliente</h5>

        </div>

        <div class="card-body">

            <p><strong>Nombre:</strong> {{ orden.cliente.nombres }}</p>

            <p><strong>Documento:</strong> {{
orden.cliente.tipo_identificacion.nombre_tipo_identificacion }} {{
orden.cliente.nro_documento }}</p>

            <p><strong>Email:</strong> {{ orden.cliente.correo_electronico
}}</p>

            {% if orden.cliente.direccion %}

            <p><strong>Dirección:</strong> {{ orden.cliente.direccion
}}</p>

            {% endif %}

        </div>

    </div>

</div>

```

```

        </div>
    </div>

    <div class="card shadow">
        <div class="card-header bg-white">
            <h5 class="mb-0">Acciones</h5>
        </div>
        <div class="card-body">
            <div class="d-grid gap-2">
                <a href="{% url 'generate_pdf_order' orden.pedido_id %}"
class="btn btn-primary" target="_blank">
                    <i class="fas fa-file-pdf me-2"></i>Descargar Factura
                </a>
                <a href="mailto:?subject=Orden%20#{{ orden.nro_pedido
}}&body=Detalles%20de%20tu%20orden%20#{{ orden.nro_pedido }}:%20A%0ATotal:%20${{
orden.importe }}" class="btn btn-outline-secondary">
                    <i class="fas fa-envelope me-2"></i>Enviar por Email
                </a>
                {% if orden.estado == 1 %}
                <a href="{% url 'cancel_order' orden.pedido_id %}"
class="btn btn-outline-danger" onclick="return confirm('¿Estás seguro de
cancelar esta orden?')">
                    <i class="fas fa-times-circle me-2"></i>Cancelar Orden
                </a>
                {% endif %}
            </div>
        </div>
    </div>

```

```
</div>

</div>

</div>

{% endblock %}
```

Ahora nos toca actualizar nuestras rutas para que podamos invocar al carrito de compras para ello vamos al archivo `core/urls.py` y agregamos lo siguiente:

Python

```
# URLs para el Carrito de Compras

path('carrito/', views.cart_detail, name='cart_detail'),

path('carrito/agregar/<uuid:articulo_id>/', views.cart_add,
name='cart_add'),

path('carrito/eliminar/<uuid:articulo_id>/', views.cart_remove,
name='cart_remove'),

path('carrito/vaciar/', views.cart_clear, name='cart_clear'),

path('checkout/', views.checkout, name='checkout'),

path('orden/<uuid:pedido_id>/', views.order_detail, name='order_detail'),

path('orden/cancelar/<uuid:pedido_id>/', views.cancel_order,
name='cancel_order'),

path('orden/pdf/<uuid:pedido_id>/', views.generate_pdf_order,
name='generate_pdf_order'),
```

Modifica el archivo `templates/core/articulos/detail.html` para incluir un botón para añadir al carrito. El fragmento de código que debemos modificar debe ser colocado después de la sección de “Acciones rápidas”

Python

```
<form action="{% url 'cart_add' articulo.articulo_id %}"
method="post" class="mb-3">

    {% csrf_token %}

    <div class="input-group mb-3">

        <span class="input-group-text">Cantidad</span>

        <input type="number" name="cantidad" value="1" min="1"
class="form-control">

        <button type="submit" class="btn btn-primary">

            <i class="fas fa-cart-plus me-1"></i>Añadir al

Carrito

        </button>

    </div>

</form>
```

Como el archivo ya existe y ya se tenía código previamente guardado, debemos asegurarnos que el archivo quede así:

```
160     <div class="card-body">
161         <form action="{% url 'cart_add' articulo.articulo_id %}" method="post" class="mb-3">
162             {% csrf_token %}
163             <div class="input-group mb-3">
164                 <span class="input-group-text">Cantidad</span>
165                 <input type="number" name="cantidad" value="1" min="1" class="form-control">
166                 <button type="submit" class="btn btn-primary">
167                     <i class="fas fa-cart-plus me-1"></i>Añadir al Carrito
168                 </button>
169             </div>
170         </form>
171     </div class="d-grid gap-2">
```

Luego nos toca configurar el uso de sesiones en Django, para ello vamos al archivo `pos_project/settings.py` y agregamos los siguientes valores:

Python

```
# Duración de la sesión (2 semanas, en segundos)
```

```
SESSION_COOKIE_AGE = 1209600

# Cerrar la sesión cuando el usuario cierra el navegador
SESSION_EXPIRE_AT_BROWSER_CLOSE = False

# Usar cookies seguros (solo transmitir por HTTPS)
SESSION_COOKIE_SECURE = False # Cambiar a True en producción con HTTPS

# Motor de almacenamiento de sesiones
SESSION_ENGINE = 'django.contrib.sessions.backends.db' # Almacenar en la base de datos
```

Paso 3: Uso de Sesiones para Historial de Navegación

Implementemos un sistema para guardar el historial de productos visitados:

Para ello vamos al archivo `core/views.py` y modificamos la vista `articulo_detail`, tal y como se muestra en el siguiente bloque de código

```
Python

@login_required
def articulo_detail(request, articulo_id):
    """Vista para ver el detalle de un artículo"""
    articulo = get_object_or_404(Articulo, articulo_id=articulo_id)

    # Guardar en el historial de productos visitados
```

```

if 'viewed_products' not in request.session:
    request.session['viewed_products'] = []

# Convertir UUID a string para poder guardarlo en sesión
producto_actual = str(articulo.articulo_id)
viewed_products = request.session['viewed_products']

# Eliminar si ya existe y añadir al principio
if producto_actual in viewed_products:
    viewed_products.remove(producto_actual)

# Añadir al principio y mantener solo los últimos 5
viewed_products.insert(0, producto_actual)
request.session['viewed_products'] = viewed_products[:5]
request.session.modified = True

# Obtener productos visitados recientemente
recent_products = []

if viewed_products:
    recent_uuids = [uuid.UUID(id_str) for id_str in viewed_products[1:6]]
# Excluir el actual

    if recent_uuids:
        recent_products =
Articulo.objects.filter(articulo_id__in=recent_uuids)

context = {

```

```

        'articulo': articulo,

        'recent_products': recent_products,
    }

    return render(request, 'core/articulos/detail.html', context)

```

Añade esto a la template de detalle (`templates/core/articulos/detail.html`):

Python

```

<!-- Añadir en templates/core/articulos/detail.html -->

{% if recent_products %}

<div class="card shadow mt-4">

    <div class="card-header bg-white">

        <h5 class="mb-0">Vistos recientemente</h5>

    </div>

    <div class="card-body">

        <div class="row">

            {% for product in recent_products %}

            <div class="col-md-3 mb-2">

                <div class="card">

                    <div class="card-body p-2">

                        <h6 class="card-title small">{{
product.descripcion|truncatechars:25 }}</h6>

                        <p class="card-text text-primary mb-1">${{
product.listaprecio.precio_1 }}</p>

```

```

                                <a href="{% url 'articulo_detail'
product.articulo_id %}" class="btn btn-sm btn-outline-primary">Ver</a>

                                </div>

                                </div>

                                </div>

                                {% endfor %}

                                </div>

                                </div>

                                </div>

                                {% endif %}

```

El bloque de código anterior debe quedar así

```

<div class="row">
  <div class="col-md-8"> ...
  </div>

  <div class="col-md-4"> ...
  </div>

  <!-- Añadir en templates/core/articulos/detail.html -->

  {% if recent_products %}
  <div class="card shadow mt-4">
    <div class="card-header bg-white">
      <h5 class="mb-0">Vistos recientemente</h5>
    </div>
    <div class="card-body">
      <div class="row">
        {% for product in recent_products %}
        <div class="col-md-3 mb-2">
          <div class="card">
            <div class="card-body p-2">
              <h6 class="card-title small">{{ product.descripcion|truncatechars:25 }}</h6>
              <p class="card-text text-primary mb-1">${{ product.listaprecio.precio_1 }}</p>
              <a href="{% url 'articulo_detail' product.articulo_id %}" class="btn btn-sm btn-outline-primary">Ver
            </div>
          </div>
        </div>
        {% endfor %}
      </div>
    </div>
  </div>
  {% endif %}
</div>

```

Paso 3: Autenticación Avanzada de Django - OAuth con Google y Facebook

Para implementar autenticación con proveedores OAuth como Google y Facebook, usaremos django-allauth. Primero, instalamos las dependencias:

Unset

```
pip install django-allauth
```

Actualiza `settings.py` para configurar django-allauth (algunas secciones deben ser añadidas otras solo actualizadas):

Python

```
INSTALLED_APPS = [

    # ...

    'django.contrib.sites',

    'allauth',

    'allauth.account',

    'allauth.socialaccount',

    'allauth.socialaccount.providers.google',

    'allauth.socialaccount.providers.facebook',

    # ...

]

MIDDLEWARE = [

    # ... middleware existentes

    'django.contrib.sites.middleware.CurrentSiteMiddleware',
```

```

]

AUTHENTICATION_BACKENDS = [

    # Needed to login by username in Django admin
    'django.contrib.auth.backends.ModelBackend',

    # `allauth` specific authentication methods
    'allauth.account.auth_backends.AuthenticationBackend',

]

SITE_ID = 1


# Auth settings

AUTH_USER_MODEL = 'accounts.Usuario' # Asegúrate de que está correctamente
configurado

LOGIN_REDIRECT_URL = 'home'

LOGOUT_REDIRECT_URL = 'login'


# Allauth settings

ACCOUNT_USER_MODEL_USERNAME_FIELD = 'username'

ACCOUNT_EMAIL_REQUIRED = True

ACCOUNT_USERNAME_REQUIRED = True

ACCOUNT_AUTHENTICATION_METHOD = 'username_email'

ACCOUNT_EMAIL_VERIFICATION = 'optional'


# Social providers settings

SOCIALACCOUNT_PROVIDERS = {

```

```
'google': {
  'APP': {
    'client_id': 'tu-client-id-de-google',
    'secret': 'tu-secret-de-google',
    'key': ''
  },
  'SCOPE': [
    'profile',
    'email',
  ],
  'AUTH_PARAMS': {
    'access_type': 'online',
  }
},
'facebook': {
  'APP': {
    'client_id': 'tu-client-id-de-facebook',
    'secret': 'tu-secret-de-facebook',
    'key': ''
  },
  'METHOD': 'oauth2',
  'SCOPE': ['email', 'public_profile'],
  'VERIFIED_EMAIL': True,
}
}
```

Actualiza el archivo principal de URLs en `pos_project/urls.py`:

Python

```
from django.urls import path, include

urlpatterns = [

    # ... URLs existentes

    path('accounts/', include('allauth.urls')), # URLs para allauth

]
```

Finalmente, adaptemos el formulario de login (`templates/accounts/login.html`) para incluir botones de autenticación social:

Python

```
<div class="text-center my-3">

    <p class="text-muted">O inicia sesión con</p>

    <div class="d-flex justify-content-center gap-2">

        <a href="{% provider_login_url 'google' %}" class="btn
btn-outline-danger">

            <i class="fab fa-google me-2"></i>Google

        </a>

        <a href="{% provider_login_url 'facebook' %}" class="btn
btn-outline-primary">

            <i class="fab fa-facebook-f me-2"></i>Facebook

        </a>

    </div>

</div>
```

Ese fragmento de código debe ser incorporado después del botón “Iniciar sesión”, entonces nuestra template debe quedar así:

```
<div class="d-grid mb-3">
  <button type="submit" class="btn btn-primary btn-lg">
    <i class="fas fa-sign-in-alt me-2"></i>Iniciar Sesión
  </button>
</div>

<div class="text-center my-3">
  <p class="text-muted">0 inicia sesión con</p>
  <div class="d-flex justify-content-center gap-2">
    <a href="{% provider_login_url 'google' %}" class="btn btn-outline-danger">
      <i class="fab fa-google me-2"></i>Google
    </a>
    <a href="{% provider_login_url 'facebook' %}" class="btn btn-outline-primary">
      <i class="fab fa-facebook-f me-2"></i>Facebook
    </a>
  </div>
</div>
```

Paso 3: Paginación en Django

El sistema de paginación es una de las características más comunes en blogs, motores de búsqueda, listas de resultados, etc. Viendo la popularidad del sistema de paginación los desarrolladores de django han construido una clase Paginator para que los desarrolladores web no tengan que pensar en la lógica para hacer paginadores. Ya hemos visto un ejemplo básico en la vista `articulos_list`. Veamos cómo implementar paginación más avanzada:

a. Paginación básica

En el ejemplo que ya tenemos en `articulos_list` usa el paginador básico de Django:

Python

```
from django.core.paginator import Paginator

def articulos_list(request):
    articulos_list = Articulo.objects.all()

    # Filtros (podrías expandir esto)
```

```

q = request.GET.get('q')

if q:
    articulos_list = articulos_list.filter(descripcion__icontains=q)

# Paginación

paginator = Paginator(articulos_list, 15) # 15 artículos por página

page_number = request.GET.get('page')

articulos = paginator.get_page(page_number)

context = {
    'articulos': articulos,
}

return render(request, 'core/articulos/list.html', context)

```

b. Paginación infinita con JavaScript

Para implementar paginación "infinita" (carga automática al hacer scroll), podemos usar JavaScript con la API paginada, para ello vamos a crear el archivo `templates/core/articulos/list_infinite.html` y escribimos lo siguiente:

Python

```

<!-- Añadir a templates/core/articulos/list_infinite.html -->

{% extends 'base.html' %}

```

```

{% block title %}Catálogo de Artículos - Sistema POS{% endblock %}

{% block content %}

<div class="d-flex justify-content-between align-items-center mb-4">

    <h2 class="h3"><i class="fas fa-boxes me-2"></i>Catálogo de Artículos</h2>

    <div>

        <input type="text" id="searchInput" class="form-control"
placeholder="Buscar artículos...">

    </div>

</div>

<div class="row" id="articulos-container">

    <!-- Aquí se cargarán los artículos -->

</div>

<div class="text-center my-4 d-none" id="loading">

    <div class="spinner-border text-primary" role="status">

        <span class="visually-hidden">Cargando...</span>

    </div>

</div>

{% endblock %}

{% block extra_js %}

<script>

    document.addEventListener('DOMContentLoaded', function() {

```

```
let nextPage = null;

let isLoading = false;

const container = document.getElementById('articulos-container');
const loadingIndicator = document.getElementById('loading');
const searchInput = document.getElementById('searchInput');

// Función para cargar artículos
async function loadArticulos(url, append = false) {
  if (isLoading) return;
  isLoading = true;

  if (!append) {
    container.innerHTML = '';
  }

  loadingIndicator.classList.remove('d-none');

  try {
    const response = await fetch(url);
    const data = await response.json();

    nextPage = data.next;

    data.results.forEach(articulo => {
      const card = document.createElement('div');
```

```

card.className = 'col-md-4 mb-4';
card.innerHTML = `
    <div class="card shadow h-100">
        <div class="card-body">
            <h5
class="card-title">${articulo.descripcion}</h5>
            <p class="card-text">
                <strong>Código:</strong>
                ${articulo.codigo_articulo}<br>
                <strong>Grupo:</strong>
                ${articulo.grupo.nombre_grupo}<br>
                <strong>Línea:</strong>
                ${articulo.linea.nombre_linea}<br>
                <strong>Stock:</strong>
                <span class="badge ${articulo.stock < 10 ?
'bg-danger' : 'bg-success'}">${articulo.stock}</span>
            </p>
            <h6
class="text-primary">${articulo.listaprecio.precio_1}</h6>
        </div>
        <div class="card-footer bg-white">
            <a
href="/core/articulos/${articulo.articulo_id}/" class="btn btn-sm
btn-outline-primary">Ver Detalles</a>
        </div>
    </div>
`;
container.appendChild(card);

```

```

    });

    } catch (error) {
        console.error('Error al cargar artículos:', error);
    } finally {
        loadingIndicator.classList.add('d-none');
        isLoading = false;
    }
}

// Cargar artículos iniciales
loadArticulos('/core/api/articulos/');

// Cargar más artículos al hacer scroll
window.addEventListener('scroll', () => {
    if (nextPage && !isLoading &&
        window.innerHeight + window.scrollY >=
document.body.offsetHeight - 200) {
        loadArticulos(nextPage, true);
    }
});

// Filtrar artículos al escribir
let searchTimeout;
searchInput.addEventListener('input', (e) => {
    clearTimeout(searchTimeout);

```

```

        searchTimeout = setTimeout(() => {
            const searchTerm = e.target.value.trim();
            const url = searchTerm
                ?
                `/core/api/articulos/?descripcion=${encodeURIComponent(searchTerm)}`
                : '/core/api/articulos/';
            loadArticulos(url);
        }, 500);
    });
</script>
{% endblock %}

```

Ahora, tenemos que añadir la URL para esta vista:

Python

```

# Añadir a core/urls.py

path('articulos/catalogo/', views.articulos_catalogo,
     name='articulos_catalogo'),

```

Y la vista correspondiente:

Python

```

# Añadir a core/views.py

def articulos_catalogo(request):

```

```
"""Vista para el catálogo con paginación infinita"""  
  
return render(request, 'core/articulos/list_infinite.html')
```

Paso 4: Django Messages Framework

El framework de mensajes de Django proporciona una forma elegante de mostrar notificaciones temporales (a menudo llamadas «mensajes flash») que persisten a través de las peticiones y desaparecen después de ser mostradas.

¿Por qué usar el framework de mensajes de Django?

Imagina un escenario en el que un usuario envía un formulario. Después de procesarlo, quieres informarles de que el envío se ha realizado correctamente o si ha habido algún problema.

En lugar de almacenar mensajes en la sesión manualmente o depender de una compleja lógica front-end, Django proporciona un sistema integrado para manejar mensajes temporales sin esfuerzo.

Principales ventajas del framework de mensajes:

- Almacena mensajes para la siguiente solicitud.
- Funciona tanto para usuarios autenticados como anónimos.
- Categoriza los mensajes en diferentes niveles (éxito, error, advertencia, etc.).
- Permite la personalización mediante CSS y etiquetas adicionales.

Tipos de Mensajes

Django proporciona varios niveles de mensajes:

- `messages.DEBUG`: Solo en desarrollo
- `messages.INFO`: Información general
- `messages.SUCCESS`: Acciones exitosas
- `messages.WARNING`: Advertencias
- `messages.ERROR`: Errores

El framework de mensajes de Django está habilitado por defecto cuando creas un nuevo proyecto usando `django-admin startproject`.

Sin embargo, para asegurarse de que está configurado correctamente, compruebe que su configuración incluye:

Python

```
INSTALLED_APPS = [  
    ...  
    'django.contrib.messages',  
]
```

Vamos a personalizar la apariencia de los mensajes, para ello vamos a ir a la plantilla base `templates/base.html` y agregamos lo siguiente:

Python

```
{% if messages %}  
    <div class="messages">  
        {% for message in messages %}  
            <div class="alert  
                {% if message.tags == 'debug' %}alert-secondary{% endif  
                {% if message.tags == 'info' %}alert-info{% endif %}  
                {% if message.tags == 'success' %}alert-success{% endif  
                {% if message.tags == 'warning' %}alert-warning{% endif  
                {% if message.tags == 'error' %}alert-danger{% endif %}  
                alert-dismissible fade show" role="alert">  
                <div class="d-flex align-items-center">
```

```

        {% if message.tags == 'debug' %}<i class="fas fa-bug
me-2"></i>{% endif %}

        {% if message.tags == 'info' %}<i class="fas fa-info-circle
me-2"></i>{% endif %}

        {% if message.tags == 'success' %}<i class="fas
fa-check-circle me-2"></i>{% endif %}

        {% if message.tags == 'warning' %}<i class="fas
fa-exclamation-triangle me-2"></i>{% endif %}

        {% if message.tags == 'error' %}<i class="fas
fa-times-circle me-2"></i>{% endif %}

        <div>{{ message }}</div>

    </div>

    <button type="button" class="btn-close" data-bs-dismiss="alert"
aria-label="Close"></button>

</div>

{% endfor %}

</div>

```

Al final nuestra plantilla base.html nos debe quedar de la siguiente forma:

```

38     {% block content %}{% endblock %}
39     {% if messages %}
40     <div class="messages">
41         {% for message in messages %}
42         <div class="alert
43             {% if message.tags == 'debug' %}alert-secondary{% endif %}
44             {% if message.tags == 'info' %}alert-info{% endif %}
45             {% if message.tags == 'success' %}alert-success{% endif %}
46             {% if message.tags == 'warning' %}alert-warning{% endif %}
47             {% if message.tags == 'error' %}alert-danger{% endif %}
48             alert-dismissible fade show" role="alert">
49             <div class="d-flex align-items-center">
50                 {% if message.tags == 'debug' %}<i class="fas fa-bug me-2"></i>{% endif %}
51                 {% if message.tags == 'info' %}<i class="fas fa-info-circle me-2"></i>{% endif %}
52                 {% if message.tags == 'success' %}<i class="fas fa-check-circle me-2"></i>{% endif %}
53                 {% if message.tags == 'warning' %}<i class="fas fa-exclamation-triangle me-2"></i>{% endif %}
54                 {% if message.tags == 'error' %}<i class="fas fa-times-circle me-2"></i>{% endif %}
55                 <div>{{ message }}</div>
56             </div>
57             <button type="button" class="btn-close" data-bs-dismiss="alert" aria-label="Close"></button>
58         </div>
59         {% endfor %}
60     </div>
61     {% endif %}
62 </main>

```

Ahora vamos a ver un ejemplo completo de uso de mensajes en una vista, para ello vamos al archivo `core/views.py` y modificamos el método `articulo_create`

Python

Añadir a `core/views.py`

```
from django.contrib import messages
```

```
@login_required
```

```
def articulo_create(request):
```

```
    if request.method == 'POST':
```

```
        form = ArticuloForm(request.POST)
```

```
        precio_form = ListaPrecioForm(request.POST)
```

```
        if form.is_valid() and precio_form.is_valid():
```

```

try:

    # Generar ID para el artículo

    articulo = form.save(commit=False)

    articulo.articulo_id = uuid.uuid4()

    articulo.save()

    # Guardar precios

    lista_precio = precio_form.save(commit=False)

    lista_precio.articulo = articulo

    lista_precio.save()

    messages.success(request, f'¡Artículo "{articulo.descripcion}"
creado correctamente!')

    return redirect('articulo_detail',
articulo_id=articulo.articulo_id)

except Exception as e:

    messages.error(request, f'Error al crear el artículo:
{str(e)}')

else:

    messages.warning(request, 'Por favor corrija los errores en el
formulario.')

else:

    form = ArticuloForm()

    precio_form = ListaPrecioForm()

context = {

    'form': form,

```

```
        'precio_form': precio_form,
    }

    return render(request, 'core/articulos/form.html', context)
```

Paso 5: Email notifications con Python

Las notificaciones son un componente clave de cualquier aplicación web moderna, ya que garantizan que los usuarios estén informados y comprometidos. Un sistema de notificaciones bien implementado puede manejar múltiples canales como alertas dentro de la aplicación, correos electrónicos y SMS, mientras adapta dinámicamente el contenido para una experiencia de usuario fluida. En esta guía, te guiaremos a través de la creación de un sistema de notificación robusto y escalable en Django.

Configuración de Email

Configuración básica en `settings.py`:

Python

Configuración de Email

EMAIL_BACKEND = 'django.core.mail.backends.smtp.EmailBackend'

EMAIL_HOST = 'smtp.gmail.com' **# O tu proveedor SMTP**

EMAIL_PORT = **587**

EMAIL_USE_TLS = True

EMAIL_HOST_USER = 'tu-email@gmail.com'

EMAIL_HOST_PASSWORD = 'tu-contraseña-o-clave-de-app'

DEFAULT_FROM_EMAIL = 'Sistema POS <tu-email@gmail.com>'

Para desarrollo, puedes usar el backend de consola:

Python

```
EMAIL_BACKEND = 'django.core.mail.backends.console.EmailBackend'
```

Enviar Emails de Confirmación de Orden del Cliente

Ahora vamos a implementar el envío de emails cuando se crea una orden, para ello vamos al archivo `core/views.py`

Python

```
from django.core.mail import send_mail, EmailMultiAlternatives
from django.template.loader import render_to_string
from django.utils.html import strip_tags
from django.conf import settings

def send_order_confirmation_email(orden):
    """
    Enviar email de confirmación de orden
    """
    subject = f'Confirmación de Orden #{orden.nro_pedido}'
    from_email = settings.DEFAULT_FROM_EMAIL
    to_email = orden.cliente.correo_electronico

    # Renderizar el cuerpo del email en HTML
    html_content = render_to_string('emails/order_confirmation.html', {
```

```
'orden': orden,
'items': orden.items_orden_compra.all(),
}))

# Versión de texto plano
text_content = strip_tags(html_content)

# Crear el mensaje
email = EmailMultiAlternatives(
    subject,
    text_content,
    from_email,
    [to_email]
)

# Adjuntar versión HTML
email.attach_alternative(html_content, "text/html")

# Enviar el email
email.send()
```

Añade esta función al proceso de checkout:

Python

```
# Modificar en checkout dentro de core/views.py

# Después de crear la orden exitosamente:

try:

    send_order_confirmation_email(orden)

except Exception as e:

    messages.warning(request, f'Orden creada pero no se pudo enviar el email de confirmación: {str(e)}')
```

El código anterior quedaría integrado a nuestro método checkout

```
# Limpiar el carrito
cart.clear()
send_order_confirmation_email(orden)
messages.success(request, f'¡Orden creada exitosamente! Tu número de orden es: {orden.nro_pedido}')
return redirect('order_detail', pedido_id=orden.pedido_id)

except Exception as e:
    messages.error(request, f'Error al procesar la orden: {str(e)}')
    messages.warning(request, f'Orden creada pero no se pudo enviar el email de confirmación: {str(e)}')
    return redirect('cart_detail')
```

Creación de Plantillas de Email Personalizadas

Creemos una plantilla HTML para los emails de confirmación de orden, para ello vamos a la carpeta template y creamos una carpeta llamada emails, y luego la template templates/emails/order_confirmation.html y escribimos lo siguiente:

Python

```
<!-- Crear archivo templates/emails/order_confirmation.html -->
```

```
<!DOCTYPE html>

<html>

<head>

  <meta charset="UTF-8">

  <title>Confirmación de Orden #{{ orden.nro_pedido }}</title>

  <style>

    body {

      font-family: Arial, sans-serif;

      line-height: 1.6;

      color: #333;

      max-width: 600px;

      margin: 0 auto;

    }

    .header {

      background-color: #007bff;

      color: white;

      padding: 20px;

      text-align: center;

    }

    .content {

      padding: 20px;

    }

    .footer {

      background-color: #f8f9fa;

      padding: 15px;
```

```
        text-align: center;

        font-size: 0.8em;

        color: #6c757d;
    }
    table {

        width: 100%;

        border-collapse: collapse;

        margin: 20px 0;
    }
    th, td {

        padding: 10px;

        border-bottom: 1px solid #ddd;

        text-align: left;
    }
    th {

        background-color: #f8f9fa;
    }
    .total {

        font-weight: bold;

        text-align: right;
    }
    .btn {

        display: inline-block;

        background-color: #007bff;

        color: white;
```

```

        padding: 10px 15px;

        text-decoration: none;

        border-radius: 4px;

        margin-top: 15px;

    }
</style>
</head>
<body>

    <div class="header">

        <h1>Confirmación de Orden</h1>

        <p>Orden #{{ orden.nro_pedido }}</p>

    </div>

    <div class="content">

        <p>Estimado/a {{ orden.cliente.nombres }},</p>

        <p>¡Gracias por tu compra! Hemos recibido tu orden y está siendo
        procesada.</p>

        <h3>Detalles de la Orden:</h3>

        <p>

            <strong>Fecha:</strong> {{ orden.fecha_pedido }}<br>

            <strong>Estado:</strong>

            {% if orden.estado == 1 %}

                Pendiente

            {% elif orden.estado == 2 %}

```

```

        Procesando

{% elif orden.estado == 3 %}

        Completada

{% else %}

        Cancelada

{% endif %}

</p>

<h3>Productos:</h3>

<table>

    <thead>

        <tr>

            <th>Producto</th>

            <th>Precio</th>

            <th>Cantidad</th>

            <th>Total</th>

        </tr>

    </thead>

    <tbody>

        {% for item in items %}

            <tr>

                <td>{{ item.articulo.descripcion }}</td>

                <td>${{ item.precio_unitario }}</td>

                <td>{{ item.cantidad }}</td>

                <td>${{ item.total_item }}</td>

            </tr>

        {% endfor %}

    </tbody>

</table>

```

```

        </tr>

        {% endfor %}

    </tbody>

    <tfoot>

        <tr>

            <td colspan="3" class="total">Total:</td>

            <td>${{ orden.importe }}</td>

        </tr>

    </tfoot>

</table>

<p>Si tienes alguna pregunta sobre tu orden, no dudes en
contactarnos.</p>

<a href="http://localhost:8000/orden/{{ orden.pedido_id }}"
class="btn">Ver Detalles de la Orden</a>

</div>

<div class="footer">

    <p>&copy; {% now "Y" %} Sistema POS. Todos los derechos reservados.</p>

    <p>Este es un correo electrónico automático, por favor no responda a
este mensaje.</p>

</div>

</body>

</html>

```

—